

Tasks for this week:

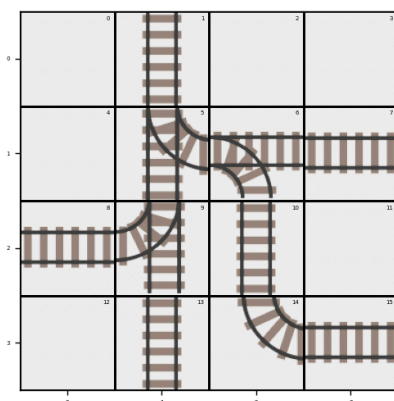
1. Test on really many trains just to see the robustness.
2. Optimize further by letting the trains only wait in one position instead of two in Roman's encoding - then recompare. **(Roman) +**
3. Implement speed (is it even required?): understand the requirements thoroughly **(Roman)**
4. Implement prioritization for trains (must not be too hard). **(Jonas)**
5. ~ Debugging tool for encodings. (first suggest)
6. Run on Mac M2 processor for comparison. **(Roman)**
7. Understand (internalize) Jonas' encoding (if it's indeed faster) like my own. **(Roman)**

13.06.25

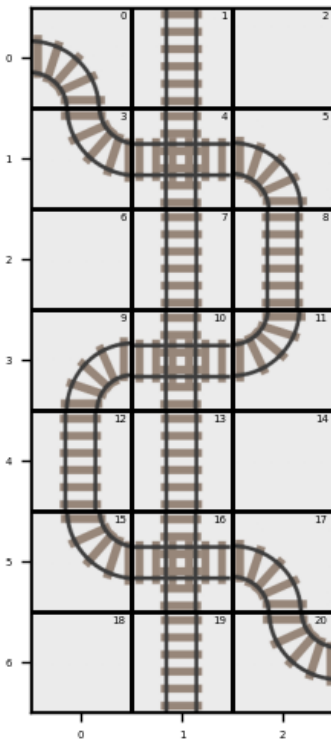
What am I to do now? Jonas claimed that his encoding proved faster on the test he ran. Perfect - I'd gladly give up mine if it is slower (it's still interesting to see why; also **there are some obvious optimization steps that I (relatively) quickly implement and recompare**). So, now, I'm down to just exploring the tests Jonas sent me:

https://docs.google.com/document/d/1117trDfRbXpntxMLkDZnqyC_I7cWz1i7/edit

Jonas' encodings are indeed much (a few times) faster. Great for us, but let's do one more attempt and implement the optimization step that I planned - only allow waiting once between two junctions, not twice (this must twice reduce the number of 'wait' atoms): let's take the smallest environment that still yields this problem:



This one doesn't yield - there are no stretches in between two junctions.



This environment (Caduceus) yields for the problem: between two junctions - cell 4 and cell 10 - the train now is allowed to stop 2 times: after the junction and before the junction.

Intuition: the cheapest way to test is to introduce one more constraint 'on top' (but not the optimal). Let's do this:

Temporary constraint to allow only one waiting instead of two:

Constraint Idea	ASP Phrasing	Code
Only one 'wait' atom must be created - right before the junction, not right after (this is arbitrary, could be otherwise).	There must be no answer set where there is more than one 'path' atom with 'wait' action between two junction cells	<pre>:- is_junction(JUNC_POS), path(IN_DIR, IN_POS, wait, IN_DIR, JUNC_POS), path(IN_DIR, JUNC_POS, wait, IN_DIR, IN_POS).</pre> Could be as simple as this, but let's test.
	OR between a junction cell and the end / start of the journey (might be two different constraints)	...

But isn't it already so given how my 'wait' atoms are created?

```
path(IN_DIR, IN_POS, wait, IN_DIR, IN_POS) :- path(IN_DIR, IN_POS, _,
OUT_DIR, OUT_POS), path(OUT_DIR, OUT_POS, _, _, _), is_junction(OUT_POS).
```

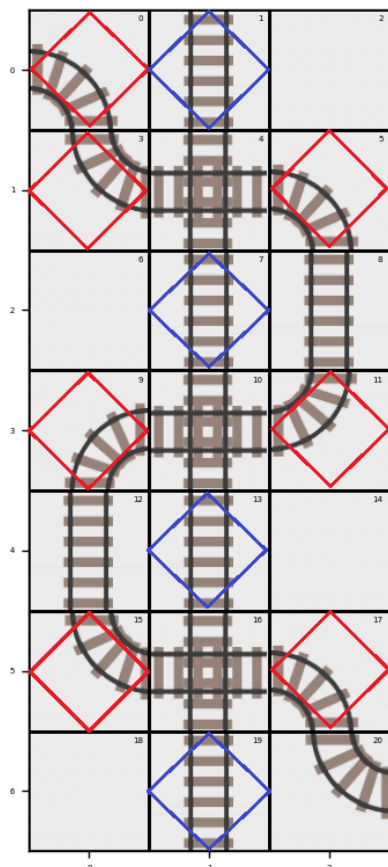
Let's see on the caduceus output:

```

path(north,19,move_forward,north,16) path(north,12,move_forward,north,9)
path(north,13,move_forward,north,10) path(north,7,move_forward,north,4)
path(north,8,move_forward,north,5) path(south,12,move_forward,south,15)
path(south,13,move_forward,south,16) path(south,7,move_forward,south,10)
path(south,8,move_forward,south,11) path(south,1,move_forward,south,4)
path(east,17,move_forward,south,20) path(east,5,move_forward,south,8)
path(east,0,move_forward,south,3) path(north,17,move_forward,west,16)
path(north,5,move_forward,west,4) path(west,9,move_forward,south,12)
path(north,9,move_forward,east,10) path(west,20,move_forward,north,17)
path(west,15,move_forward,north,12) path(west,3,move_forward,north,0)
path(south,15,move_forward,east,16) path(south,3,move_forward,east,4)
path(east,11,move_forward,north,8) path(south,11,move_forward,west,10)
path(north,16,move_forward,north,13) path(north,10,move_forward,north,7)
path(north,4,move_forward,north,1) path(south,16,move_forward,south,19)
path(south,10,move_forward,south,13) path(south,4,move_forward,south,7)
path(east,16,move_forward,east,17) path(east,10,move_forward,east,11)
path(east,4,move_forward,east,5) path(west,16,move_forward,west,15)
path(west,10,move_forward,west,9) path(west,4,move_forward,west,3)
path(south,1,wait,south,1) path(east,0,wait,east,0) path(north,19,wait,north,19)
path(north,13,wait,north,13) path(north,7,wait,north,7) path(south,13,wait,south,13)
path(south,7,wait,south,7) path(south,15,wait,south,15) path(north,9,wait,north,9)
path(south,3,wait,south,3) path(north,17,wait,north,17) path(south,11,wait,south,11)
path(north,5,wait,north,5)

```

Looks like there are more 'wait' atoms then there would've been if waiting was only allowed once per stretch. Let's check with the image:



There's definitely too much waiting here. Let's prohibit waiting more than once in between two junctions.

SUSPICION: the waiting positions now are direction specific. If yes, there's nothing to cut more (except maybe the start / end waiting, but it's really negligible).

CONCLUSION 1: there's no further meaningful optimization by reducing the waiting time - a train is already only allowed to wait right before the junction.

CONCLUSION 2: there's most likely no further improvement in terms of speed for the approach I've taken from the beginning. Any further improvement would mean starting from scratch (graph-based representation seems the most appealing). Hence, let's get to the other tasks planned for this week.

What should I do now? Now I should proceed to understanding Jonas encoding to the point that I can keep working on it to implement new features.